অবশ্যই। নিচে **শুধু Basic Definition + ছোট Graphical Diagram** দিলাম—exam-এর আগে দ্রুত revision করার জন্য। বড় explanation বাদ দিলাম।

OS Memory Management — Basic Definitions with Diagram

1. Fragmentation

Definition:
Fragmentation is the wastage of memory space caused by memory being divided into small or unusable portions.

বাংলা: Memory-এর কিছু জায়গা ব্যবহার না হওয়াকে Fragmentation বলে।

```
+------+-------+------+-------+------+
|  P1  | FREE  |  P2  | FREE  |  P3  |
+------+-------+------+-------+------+
```

2. External Fragmentation

Definition:
External fragmentation occurs when sufficient total free memory exists, but it is scattered into non-contiguous blocks.

বাংলা: মোট free memory যথেষ্ট থাকলেও contiguous না থাকলে External Fragmentation হয়।

```
+------+-------+------+-------+------+
|  P1  | FREE  |  P2  | FREE  |  P3  |
+------+-------+------+-------+------+

        Free         +         Free
         ↓                      ↓
       Scattered / Non-contiguous
```

3. Internal Fragmentation

Definition:
Internal fragmentation occurs when unused memory exists inside an allocated memory block.

বাংলা: Allocated block-এর ভিতরে unused space থাকলে Internal Fragmentation হয়।

```
+---------------------------+
| Process = 7 KB | Waste 3 KB |
+----------------------------+
                    ↑
             Internal
           Fragmentation
```

4. Compaction

Definition:
Compaction is the process of moving processes together to combine scattered free spaces into one large contiguous block.

বাংলা: ছড়িয়ে থাকা free space-গুলোকে এক জায়গায় এনে বড় একটি block তৈরি করাকে Compaction বলে।

```
Before:
+----+------+----+------+----+
| P1 | FREE | P2 | FREE | P3 |
+----+------+----+------+----+

After:
+----+----+----+-------------+
| P1 | P2 | P3 |     FREE    |
+----+----+----+-------------+
```

Dynamic Memory Allocation

5. First-Fit

Definition:
First-fit allocates the **first hole that is large enough** for the process.

বাংলা: শুরু থেকে search করে প্রথম suitable hole-এ process বসায়।

```
H1      H2       H3
100     500      300 KB
         ↑
    First suitable
```

6. Next-Fit

Definition:
Next-fit starts searching from the **location where the previous search ended**.

বাংলা: আগের allocation যেখানে শেষ হয়েছিল, সেখান থেকে search শুরু করে।

```
H1 → H2 → H3 → H4 → H5
           ↑
      Last position
           ↓
       Search →
```

7. Best-Fit

Definition:
Best-fit allocates the **smallest hole that is large enough**.

বাংলা: Suitable hole-গুলোর মধ্যে সবচেয়ে ছোট hole নির্বাচন করে।

```
100 KB    500 KB    300 KB    600 KB
                      ↑
              Best Fit
```

8. Worst-Fit

Definition:
Worst-fit allocates the **largest available hole**.

বাংলা: সবচেয়ে বড় available hole-এ process বসায়।

```
100 KB    500 KB    300 KB    600 KB
                                ↑
                         Worst Fit
```

Segmentation & Paging

9. Segmentation

Definition:
Segmentation divides a program into **variable-sized logical segments**.

বাংলা: Program-কে Code, Data, Stack ইত্যাদি logical এবং variable-size অংশে ভাগ করাকে Segmentation বলে।

```
             Program
                |
      +---------+---------+
      |         |         |
     Code      Data     Stack
    Segment   Segment   Segment
```

Address:

```
< Segment Number, Offset >
```

---

10. Segment Table

Definition:
A segment table stores the **base address and limit** of each segment.

```
+---------+---------+-------+
| Segment |  Base   | Limit |
+---------+---------+-------+
|   0     |  1000   |  400  |
|   1     |  5000   |  600  |
|   2     |  8000   |  300  |
+---------+---------+-------+
```

11. Paging

Definition:
Paging divides logical memory into fixed-size **pages** and physical memory into fixed-size **frames**.

বাংলা: Logical memory-কে Page এবং Physical memory-কে Frame-এ fixed size-এ ভাগ করাকে Paging বলে।

```
Logical Memory          Physical Memory

+-------+               +------+
| Page0| ──────────────→ |Frame3|
+-------+               +------+
| Page1| ──────────────→ |Frame0|
+-------+               +------+
| Page2| ──────────────→ |Frame5|
+-------+               +------+
```

Important:

```
Page Size = Frame Size
```

12. Page

Definition:
A page is a fixed-size block of **logical/virtual memory**.

```
Logical Memory
+------+------+
| P0   | P1   |
+------+------+
```

13. Frame

Definition:
A frame is a fixed-size block of **physical memory (RAM)**.

```
Physical Memory
+------+------+
| F0   | F1   |
+------+------+
```

14. Address Translation in Paging

Definition:
Address translation converts a logical address into a physical address using the page table.

```
CPU
 |
 | <Page, Offset>
 ↓
Page Table
 |
 | Page → Frame
 ↓
<Frame, Offset>
 |
 ↓
Physical Memory
```

Key:

```
Page Number → Frame Number
Offset      → Same
```

15. Two-Memory-Access Problem

Definition:
In paging, two memory accesses may be required: one for the page table and another for the actual data.

```
CPU
 |
 ↓
Page Table       ← 1st Access
 |
 ↓
Physical Memory  ← 2nd Access
 |
 ↓
Data
```

16. TLB

Definition:
TLB (Translation Lookaside Buffer) is a small, fast cache that stores recently used page-to-frame mappings.

```
          +----------+
CPU ────→ |   TLB    |
          +----------+
               |
           Hit ↓
```

```
              Frame
                |
                ↓
              Memory
```

17. TLB Hit

Definition:
A TLB hit occurs when the required page number is found in the TLB.

```
CPU → TLB → HIT ✓ → Frame → Memory
```

18. TLB Miss

Definition:
A TLB miss occurs when the required page number is not found in the TLB.

```
CPU → TLB → MISS ✗
                |
                ↓
           Page Table
                |
                ↓
              Frame
```

19. Effective Access Time (EAT)

Definition:
EAT is the average memory access time considering both TLB hits and misses.

Formula:

[
EAT=\alpha(\epsilon+m)+(1-\alpha)(\epsilon+2m)
]

```
                 EAT
                  |
       +--------+--------+
       |                 |
    TLB Hit          TLB Miss
       |                 |
   ε + m             ε + 2m
```

Advanced Paging

20. Hierarchical Paging

Definition:
Hierarchical paging divides a large page table into multiple levels.

```
Logical Address
       |
       ↓
Outer Page Table
       |
       ↓
Inner Page Table
       |
       ↓
Frame
       |
       ↓
Memory
```

21. Hashed Page Table

Definition:
A hashed page table uses a hash function on the virtual page number to find the corresponding page-table entry.

```
Virtual Page
      |
      ↓
Hash Function
      |
      ↓
Hash Table
      |
      ↓
Frame Number
```

22. Inverted Page Table

Definition:
An inverted page table has **one entry for each physical frame** rather than for each virtual page.

```
Physical Frames

Frame 0 → Entry
Frame 1 → Entry
Frame 2 → Entry
Frame 3 → Entry
    ↓
One Entry per Frame
```

Sharing & Protection

23. Memory Protection

Definition:
Memory protection prevents unauthorized access to memory by using access-control bits such as Read, Write, and Execute.

```
Page
 |
 ↓
Protection Check
 |
 +---- Allowed → Memory ✓
 |
 +---- Denied  → Trap ✗
```

24. Shared Pages

Definition:
Shared pages allow multiple processes to map their logical pages to the same physical frame.

```
Process P1 ──→ Page Table ──┐
                            ↓
                         Frame
                            ↑
Process P2 ──→ Page Table ──┘
```

25. Combined Segmentation and Paging

Definition:
Combined segmentation and paging divides a program into logical segments and then divides each segment into fixed-size pages.

```
Program
   |
   ↓
Segments
   |
   ↓
Pages
   |
   ↓
Frames
   |
   ↓
Physical Memory
```

Address:

```
< Segment, Page, Offset >
```

⭐ Important Differences

26. Internal vs External Fragmentation

Feature	Internal	External
Location	Inside allocated block	Outside allocated blocks
Block	Allocated block is larger	Free blocks are scattered
Main problem	Wasted space inside	Non-contiguous free space
Example	7 KB used in 10 KB block	10 KB + 8 KB free but need 20 KB

```
Internal:                External:

+--------------+         +----+----+----+----+
| Used  |Waste |         | P1 | F  | P2 | F  |
+--------------+         +----+----+----+----+
     ↑                        ↑         ↑
  Inside                    Scattered
```

27. Compaction vs Context Switching

Feature	Compaction	Context Switching
Area	Memory Management	Process Management
Purpose	Reduce external fragmentation	Switch CPU between processes
What changes?	Process memory locations	CPU execution state
Main operation	Move/relocate processes	Save & restore context
RAM movement	Yes	Normally no

```
Compaction:

P1 | F | P2 | F | P3
        ↓
P1 | P2 | P3 |   FREE


Context Switching:

CPU → P1
       ↓ Save Context
       ↓ Load P2
CPU → P2
```

28. Segmentation vs Paging

Feature	Segmentation	Paging
Unit	Segment	Page
Size	Variable	Fixed
Based on	Logical view	Physical memory management
Address	Segment + Offset	Page + Offset
Fragmentation	External	Internal
Visibility	Programmer-visible concept	Generally invisible
Mapping	Segment → Memory	Page → Frame

```
Segmentation:

&lt;Segment, Offset&gt;
       ↓
 Segment Table
       ↓
 Physical Memory


Paging:

&lt;Page, Offset&gt;
       ↓
 Page Table
       ↓
&lt;Frame, Offset&gt;
       ↓
 Physical Memory
```

---

29. SCAN / Elevator Algorithm

Definition:
SCAN is a disk scheduling algorithm in which the disk head moves in one direction, services requests, reaches the end, and then reverses direction.

```
0 ------------------------------------ 199
       20   40   60   80   100
                  ↑
              Head

40 → 60 → 80 → 100 → 199
                       ↓
199 → 100 → 80 → 60 → 40 → 20
```

Memory Trick:
SCAN = Go → Serve → End → Reverse → Serve